

上机实验1：流水线性能分析 I

1. 实验目的

- (1)加深对计算机流水线基本概念的理解;
- (2)理解 MIPS 结构如何用 5 段流水线来实现, 理解各段的功能和基本操作;
- (3)加深对数据冲突、结构冲突、控制冲突的理解, 并能够分析这些冲突对 CPU 性能的影响;
- (4)进一步理解解决数据冲突的方法, 掌握如何应用定性技术来减少数据冲突引起的流水线停顿。

2. 实验平台

使用教材附带的流水线模拟器 MIPSsim 模拟器

3. 实验内容与步骤

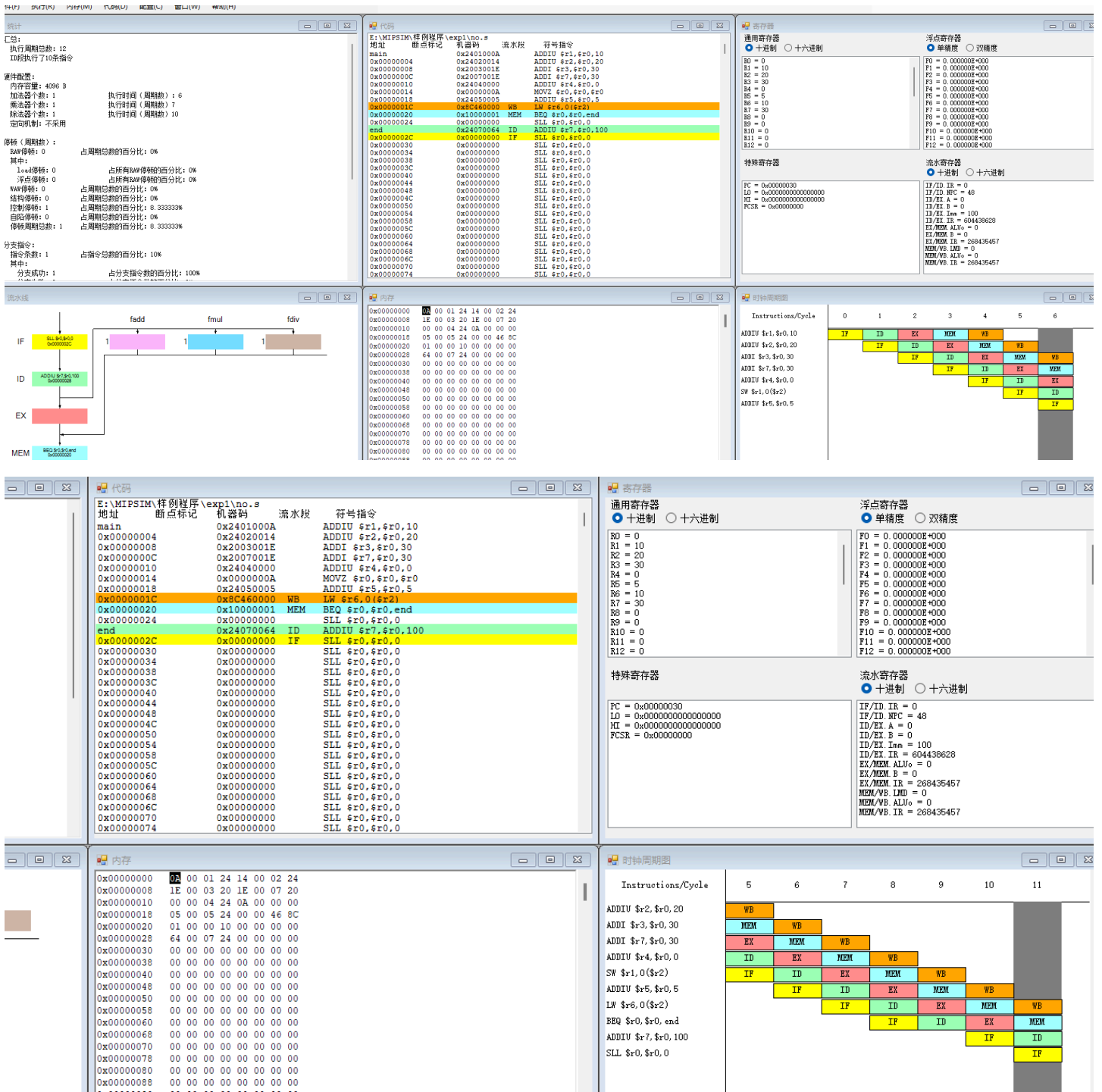
- (1)至少实现以下功能:
 - A. 能够模拟 MIPS 的 5 段流水线, 以课堂讲授为参考;
 - B. 熟悉图形交互或者命令交互;
 - C. 熟悉单步执行 (一次一个时钟周期)、执行到断点、执行到程序结束;
 - D. 掌握流水线各个段、寄存器状态的查看;
 - E. 掌握是否使用定向路径的功能选项;
 - F. 提供程序执行后的性能统计分析;
 - G. 按照 MIPS 语法, 至少支持 load、store、add、beqz 操作;
 - H. 要执行的程序可以直接输入, 或通过文件载入的方式运行;
- (2)使用在(1)中 G 提到的指令, 设计至少 3 种不同的代码组合, 实现以下功能的演示:
 - A. 没有任何冲突的流水线场景;
 - B. 有至少一次的 RAW 冲突;
 - C. 有至少一次的分支跳转;
 - D. 使用课堂讲授的内容, 对上述每一种场景进行分析

4. 实验结果

A. 没有任何冲突的流水线场景

```
.text
main:
    ADDIU $r1, $r0, 10      # $r1 = 10
    ADDIU $r2, $r0, 20      # $r2 = 20
    ADD   $r3, $r0, 30      # $r3 = 30
    ADD   $r7, $r0, 30      # $r7 = 30
    ADDIU $r4, $r0, 0x100    # $r4 = 地址0x100
    SW    $r1, 0($r2)        # 存储$r1到0x20 (SW在ADD后, 间隔足够)
    ADDIU $r5, $r0, 5        # 无关指令
    LW    $r6, 0($r2)        # 加载数据到$r6 (SW已写入内存)
    BEQZ  $r0, end          #
    NOP                                # 延迟槽
end:
    ADDIU $r7, $r0, 100      # 无冲突

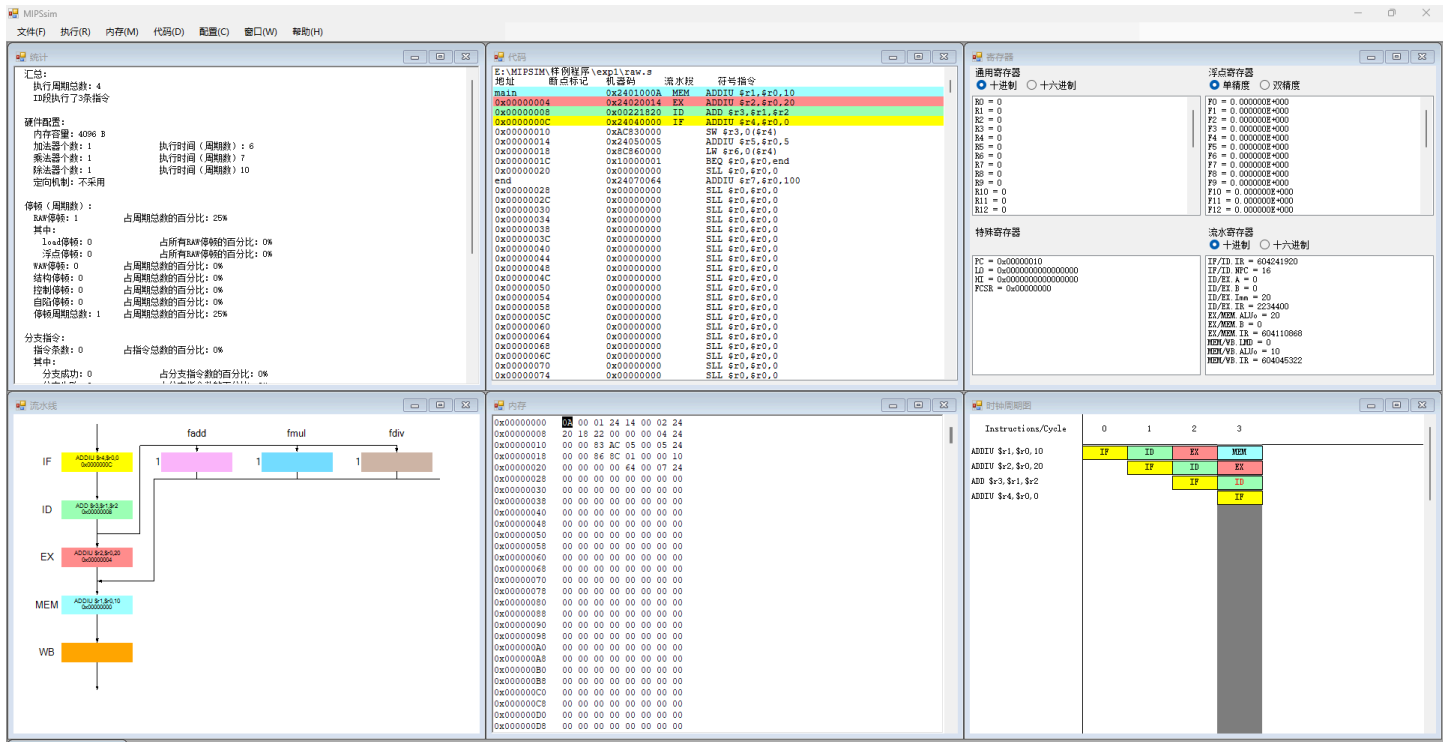
.data
    .align 2
```



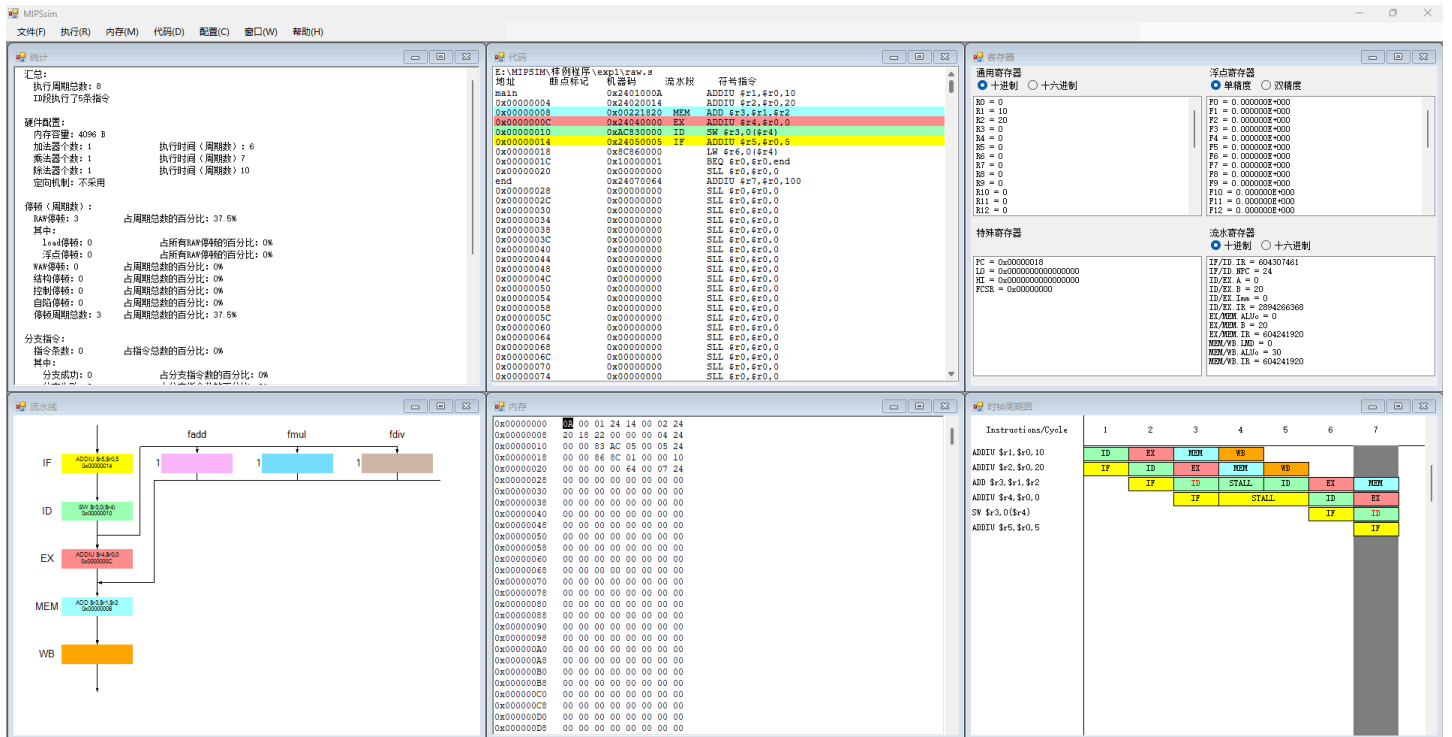
可以看到没有发生任何冲突，指令依次进入流水线，没有插入任何stall。

B. 有至少一次的RAW冲突

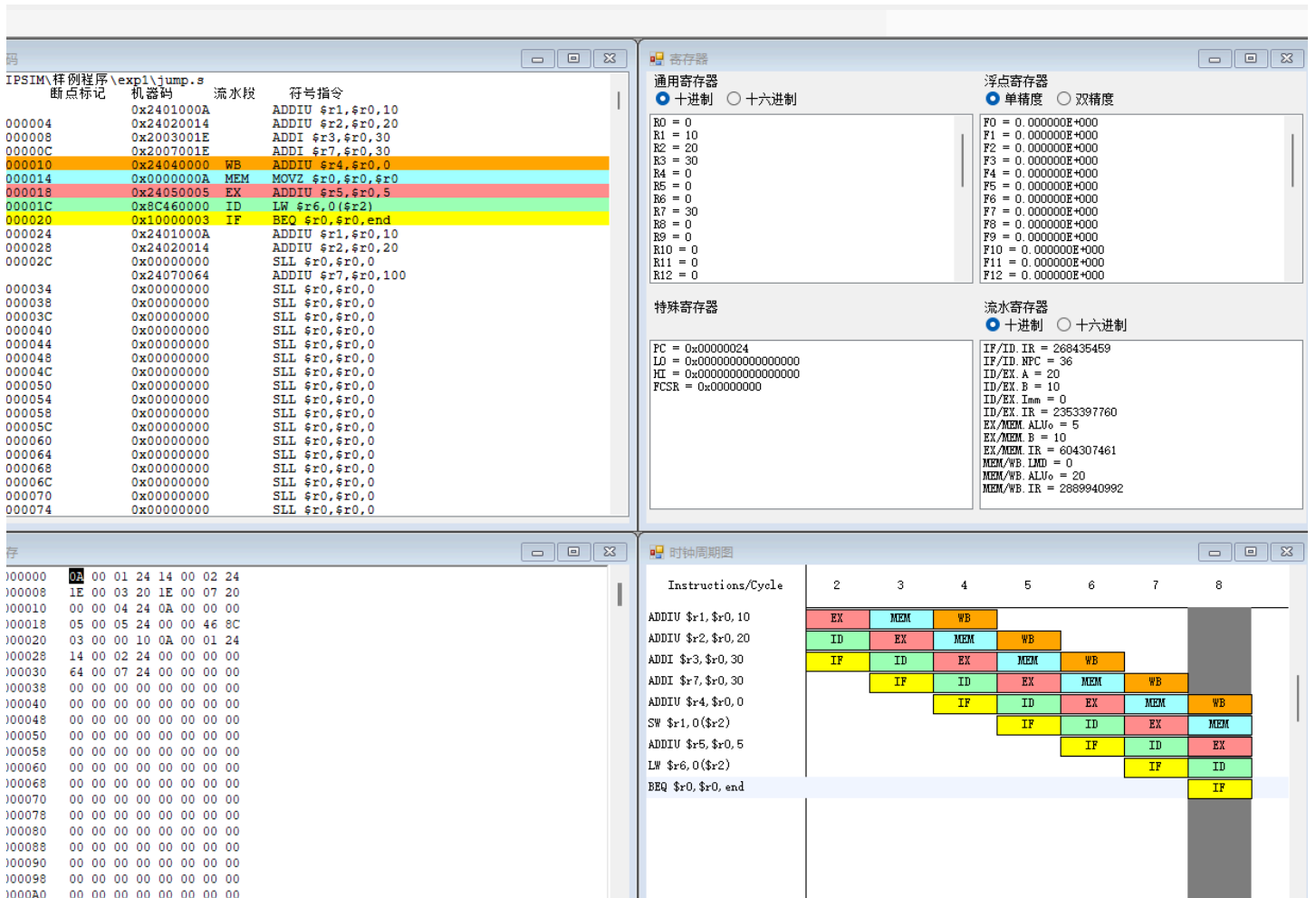
```
.text
main:
    ADDIU $r1, $r0, 10      # $r1 = 10
    ADDIU $r2, $r0, 20      # $r2 = 20
    ADD  $r3, $r1, $r2      # $r3 = 30
    ADDIU $r4, $r0, 0x100   # $r4 = 地址0x100
    SW   $r3, 0($r4)        # 存储$r3到0x100 (SW在ADD后，间隔足够)
    ADDIU $r5, $r0, 5       # 无关指令
    LW   $r6, 0($r4)        # 加载数据到$r6 (SW已写入内存)
    BEQZ $r0, end           #
    NOP                     # 延迟槽
end:
    ADDIU $r7, $r0, 100    # 无冲突
.data
.align 2
```



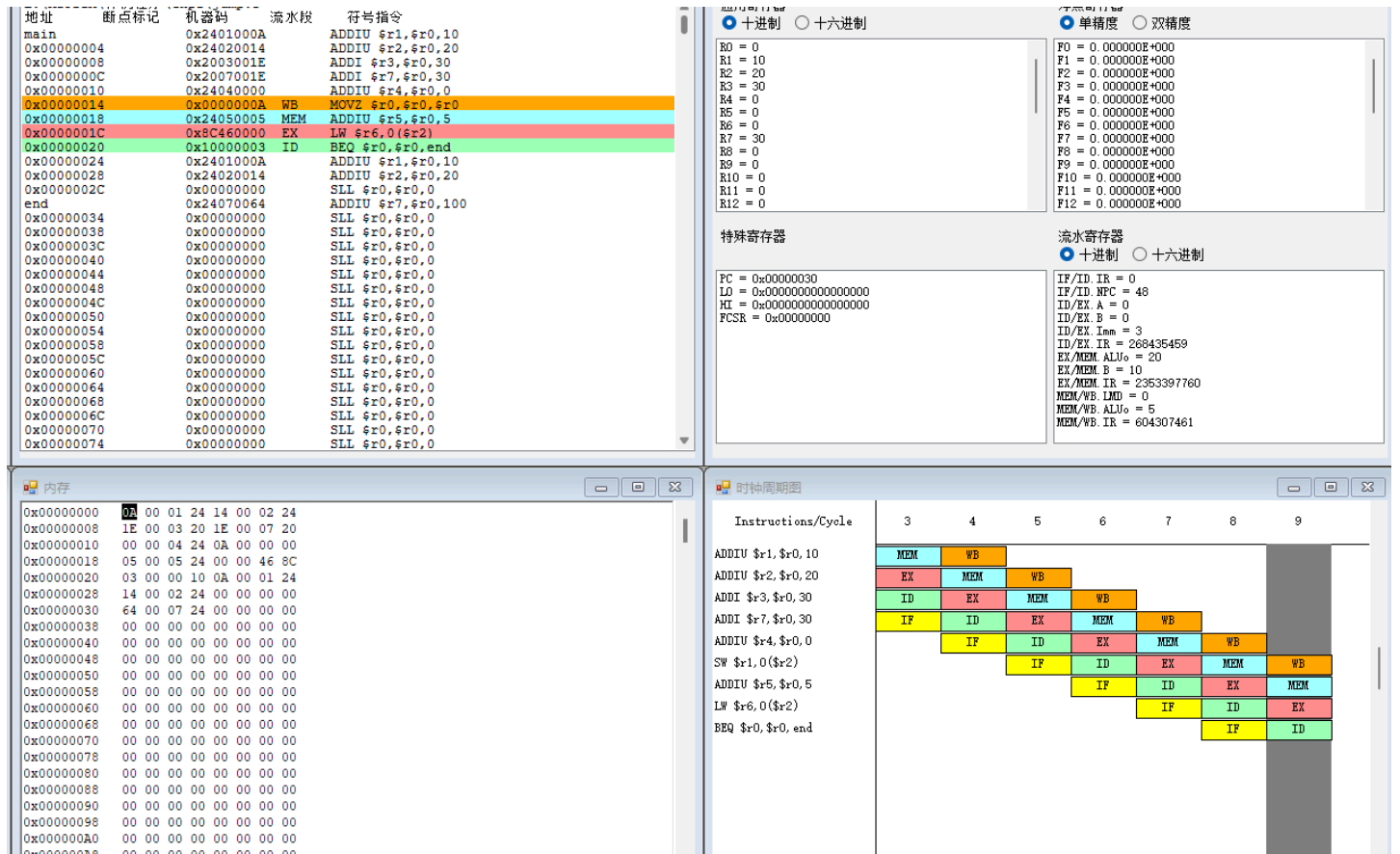
如图所示，在ADD $r3, r1, r2$ 的ID段要读取 $r1$ 、 $r2$ 寄存器的内容，此时ADDIU $r1, r0, 10$ 还在MEM段尚未写入 $r1$ ，ADDIU $r2, \$r0, 20$ 只执行到了EX段，同样未写入 $r2$ ，观察 $r1$ 、 $r2$ 的寄存器内容发现确实如此。因此发生了RAW冲突，插入了气泡。



同样地，在SW $r3, 0(r4)$ 中的ID段要读取 $r4, r3$ 的内容，而此时 $r3, r4$ 尚未写入，发生RAW冲突，插入了气泡。



BEQ的IF段，还不知道是分支指令，正常PC+4



BEQ的ID段，发现是分支指令，判断跳转成功，计算出跳转指令地址，放入PC

